

Module 2.1 – Data Warehouse Design

Logical & Physical Design

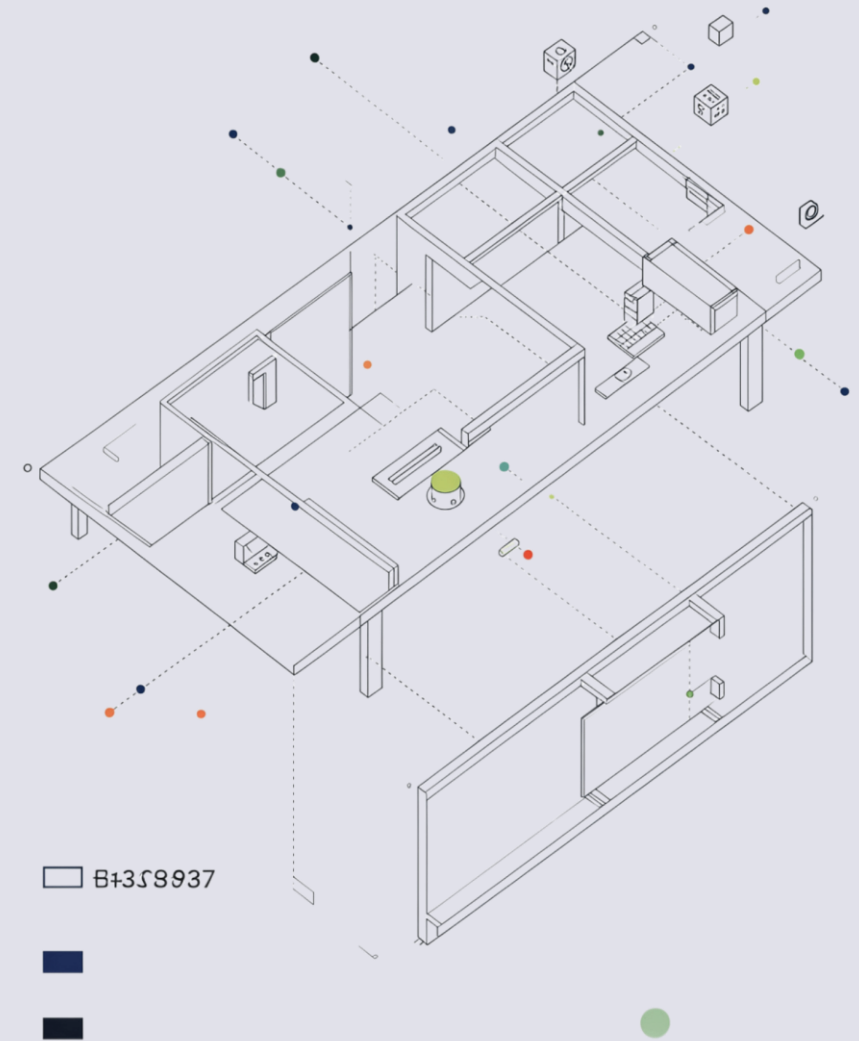
Blueprint thinking before implementation

Dimensional Modeling

Star · Snowflake · Constellation schemas

Data Cube & OLAP

Multidimensional analysis · DMQL · Concept Hierarchies



Logical Design vs Physical Design

"When designing a data warehouse, what should be planned first?"

The Architect's Analogy

Logical Design is the **blueprint** — what data exists, how it relates, and what business questions it must answer. It is entirely independent of technology.

Physical Design is the **actual building** — how data is stored, indexed, partitioned, and optimised on real hardware.

Aspect	Logical Design	Physical Design
Focus	Business entities & relationships	Storage structures & performance
Components	Entities, attributes, relationships	Tables, indexes, partitions, clusters
Output	ER diagram / Dimensional model	DDL scripts, physical schema
Example	Define Patient, Visit, Doctor entities	Create indexed tables in Oracle/SQL Server
Responsibility	Business analyst + Data architect	DBA + Data engineer

You never lay bricks before the architect draws. Changing logical design after physical build = demolishing a building to move a wall.

Running Example

Hospital Data Warehouse — Logical Before Physical

Stage 1 — Logical Model

Identify entities: **Patient, Doctor, Department, Diagnosis, Treatment, Billing**. Define relationships: a Patient has many **Visits**; each Visit links to a Doctor and a Diagnosis.

Stage 2 — Physical Schema

Decide partitioning by year, indexing on **PatientID**, and storage engine (columnar vs row-based). Works whether you deploy on Oracle, Snowflake, or Amazon Redshift.



Key rule: Logical design is technology-agnostic. Implementation details change with hardware upgrades; the logical model captures permanent business intent.

What is a Design Methodology?

A **Data Warehouse Design Methodology** is a structured approach that defines **how** a **Data Warehouse** should be planned, designed, and implemented.

Methodology	Developed First	Philosophy
Top-Down	Enterprise Data Warehouse	Organization First
Bottom-Up	Departmental Data Marts	Department First
Hybrid	Combination of both	Balance of Speed and Integration

What is a Design Methodology?

A **Data Warehouse Design Methodology** is a structured approach that defines **how a Data Warehouse should be planned, designed, and implemented.**

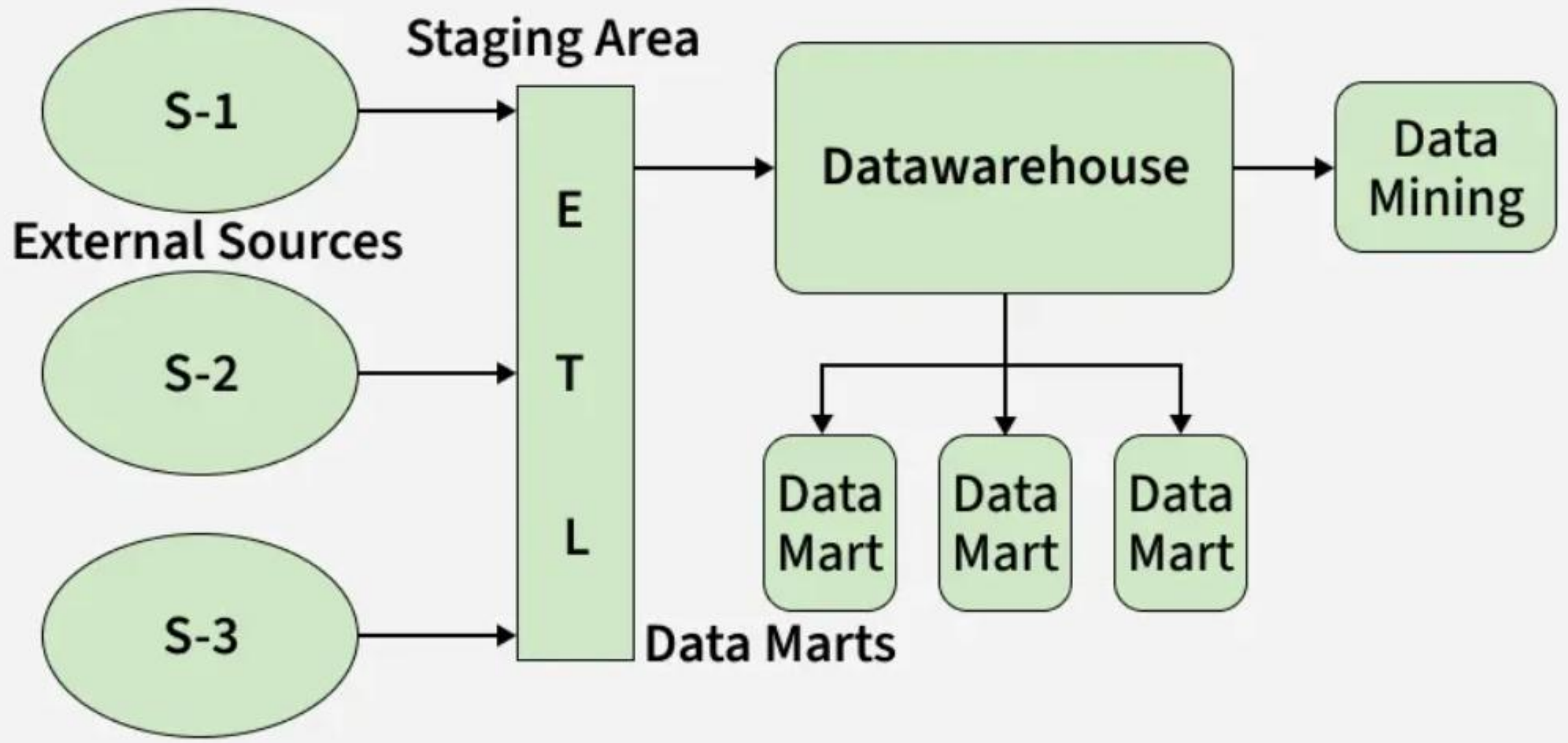
Top-Down Approach (Bill Inmon) Development starts at the **enterprise level (top).**

- First, an **Enterprise Data Warehouse (EDW)** is designed and built.
- After the EDW is complete, **department-specific Data Marts** are created from it.
- Therefore, the design process moves from the **top (organization)** to the **bottom (departments).**

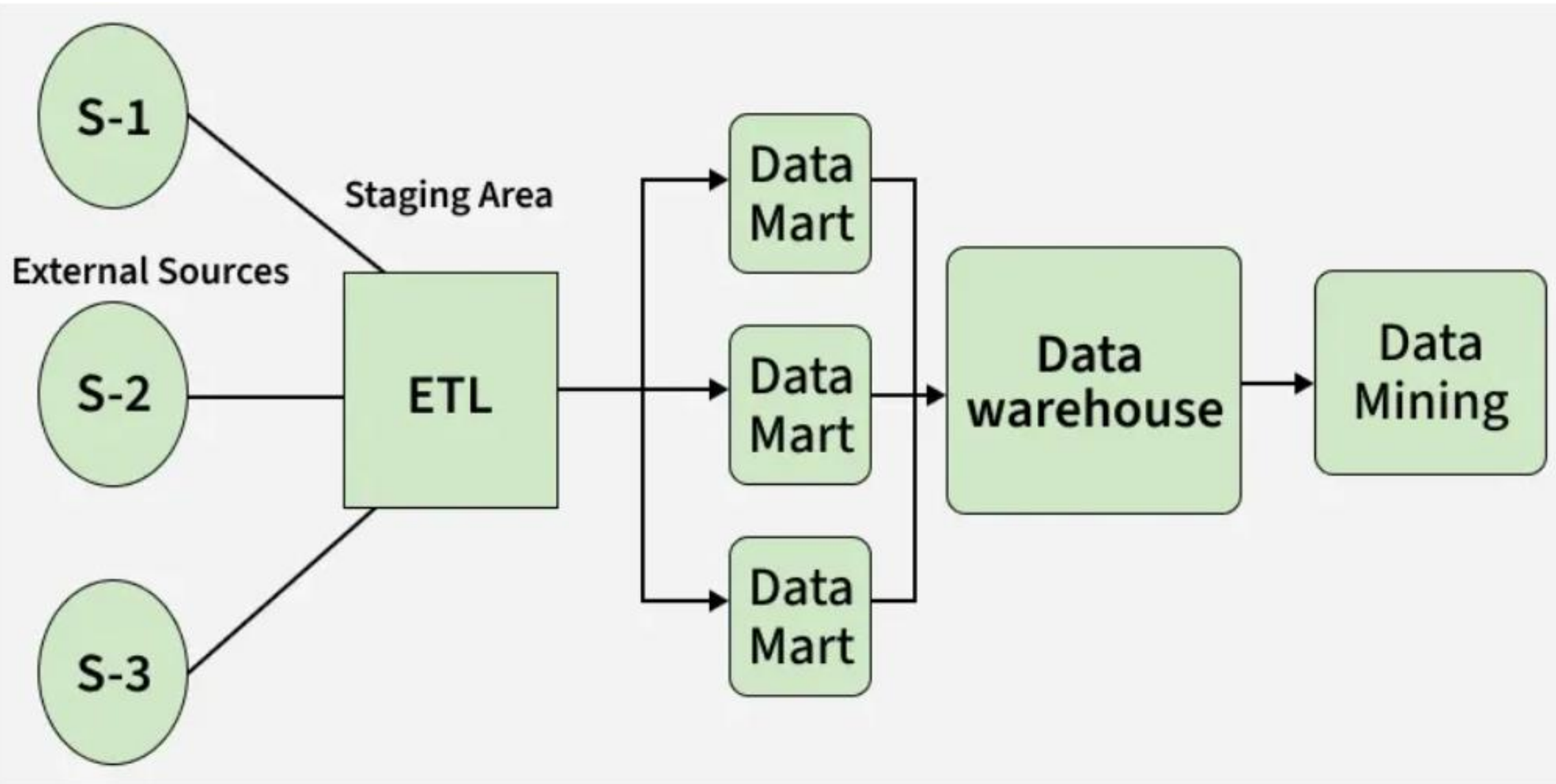
Bottom-Up Approach (Ralph Kimball) Development starts with **individual departmental Data Marts** (Sales, HR, Finance, etc.).

Later, these Data Marts are integrated to form the Enterprise Data Warehouse.

Hence, the design process moves from the **bottom (departments)** to the **top (enterprise).**



Top-Down Approach

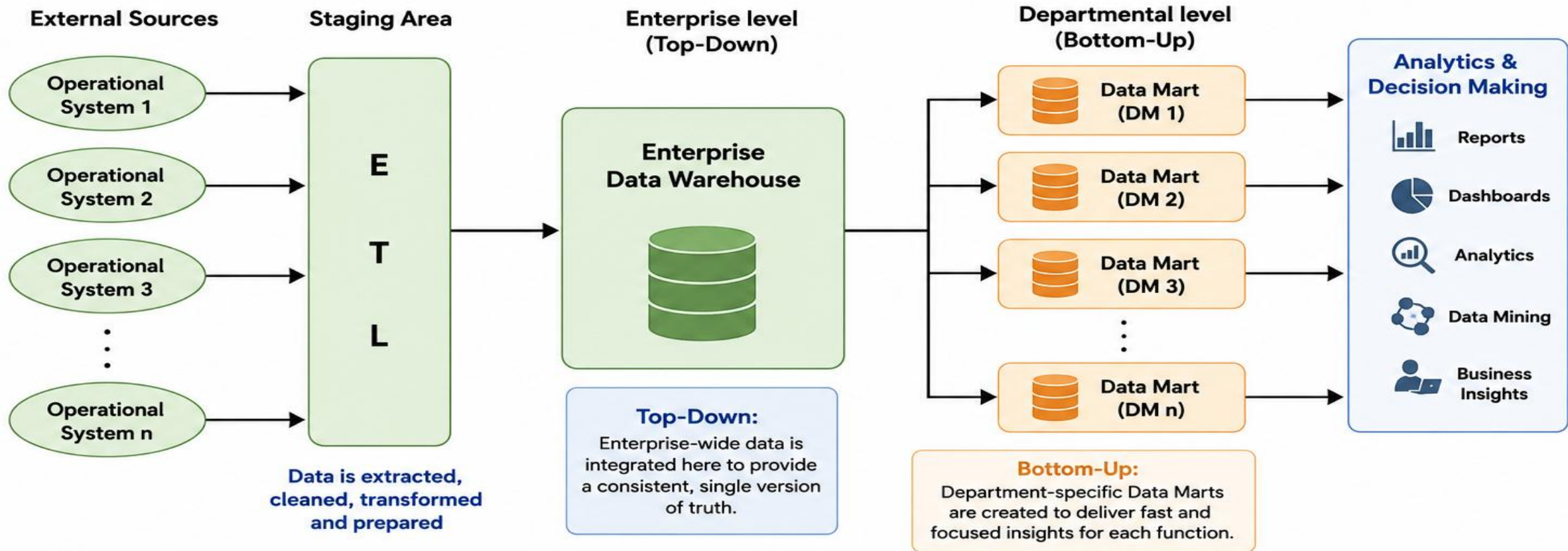


Bottom-Up Approach

Hybrid Approach

(Combination of Top-Down and Bottom-Up Approaches)

Build an enterprise Data Warehouse first (top-down) to get a consistent, integrated view of the organization, and then create department-specific Data Marts from it (bottom-up) to deliver quick, focused value.



Why it is called Hybrid?

- **Top-Down part:** The enterprise Data Warehouse is built first to ensure enterprise consistency and integration.
- **Bottom-Up part:** Data Marts are built from the EDW to meet specific departmental needs quickly.
- **Combines the strengths of both:** enterprise-wide consistency + faster departmental delivery.

Example: Flipkart

- EDW contains all sales, customer, product, inventory, payment, delivery data.
- Data Marts: Sales DM, Inventory DM, Customer DM, Finance DM, etc.
- Helps Flipkart make better decisions quickly at both enterprise and department levels.

Developing a Data Warehouse — The Roadmap

Why do we need a roadmap?

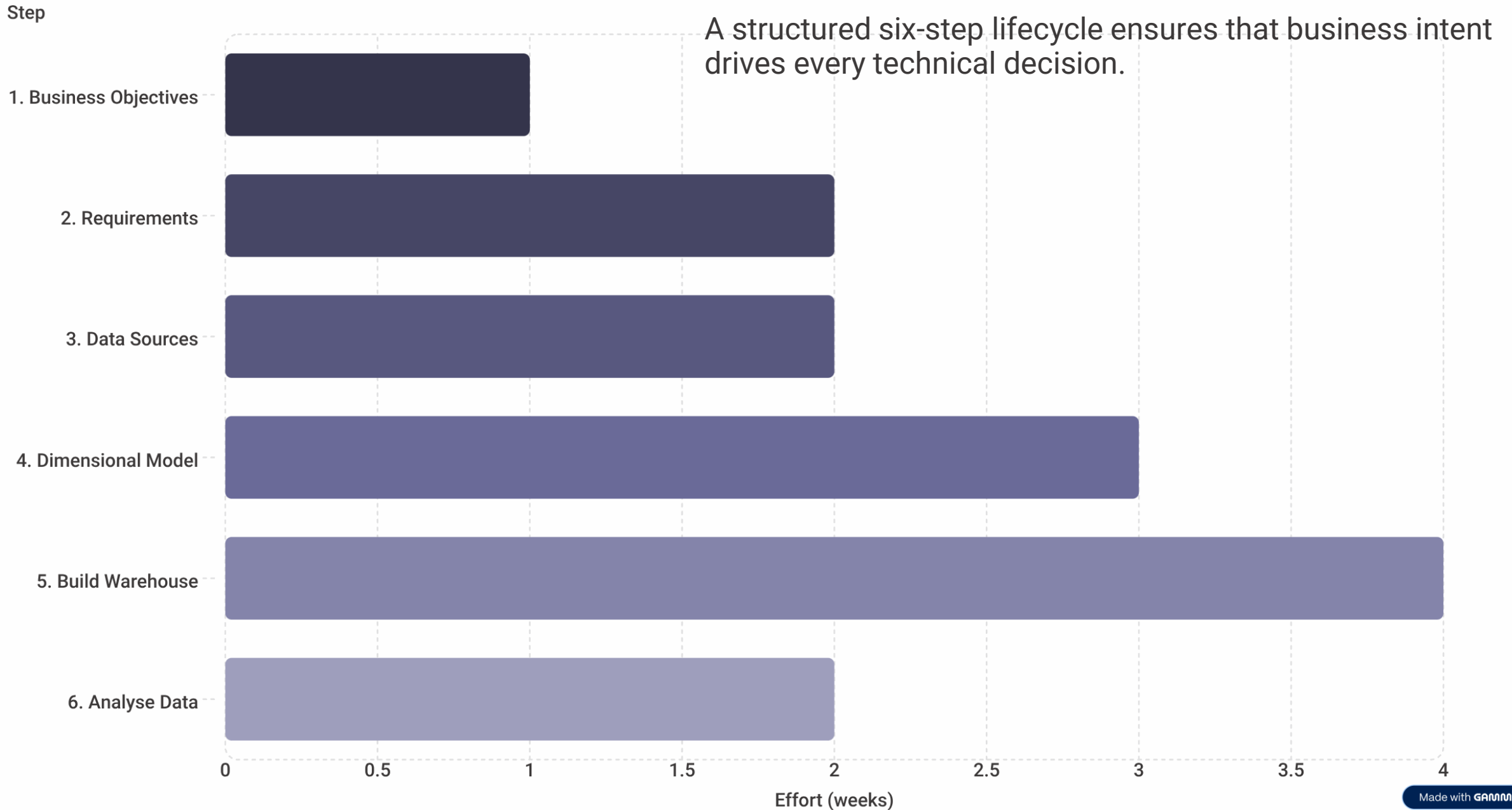
- Without a proper roadmap:
- Wrong business requirements may be collected.
- Data from different systems may not integrate correctly.
- Reports may become slow or inaccurate.
- The warehouse may not meet future business needs.
- Therefore, every organization follows a structured development process.

Developing a Data Warehouse — The Roadmap

Developing a Data Warehouse is a **systematic process of converting raw data from multiple sources into a centralized repository** that helps organizations analyze information and make better business decisions.

Instead of building the warehouse randomly, organizations follow a **step-by-step roadmap** to ensure that the warehouse meets business requirements, stores high-quality data, and supports fast reporting and analytics.

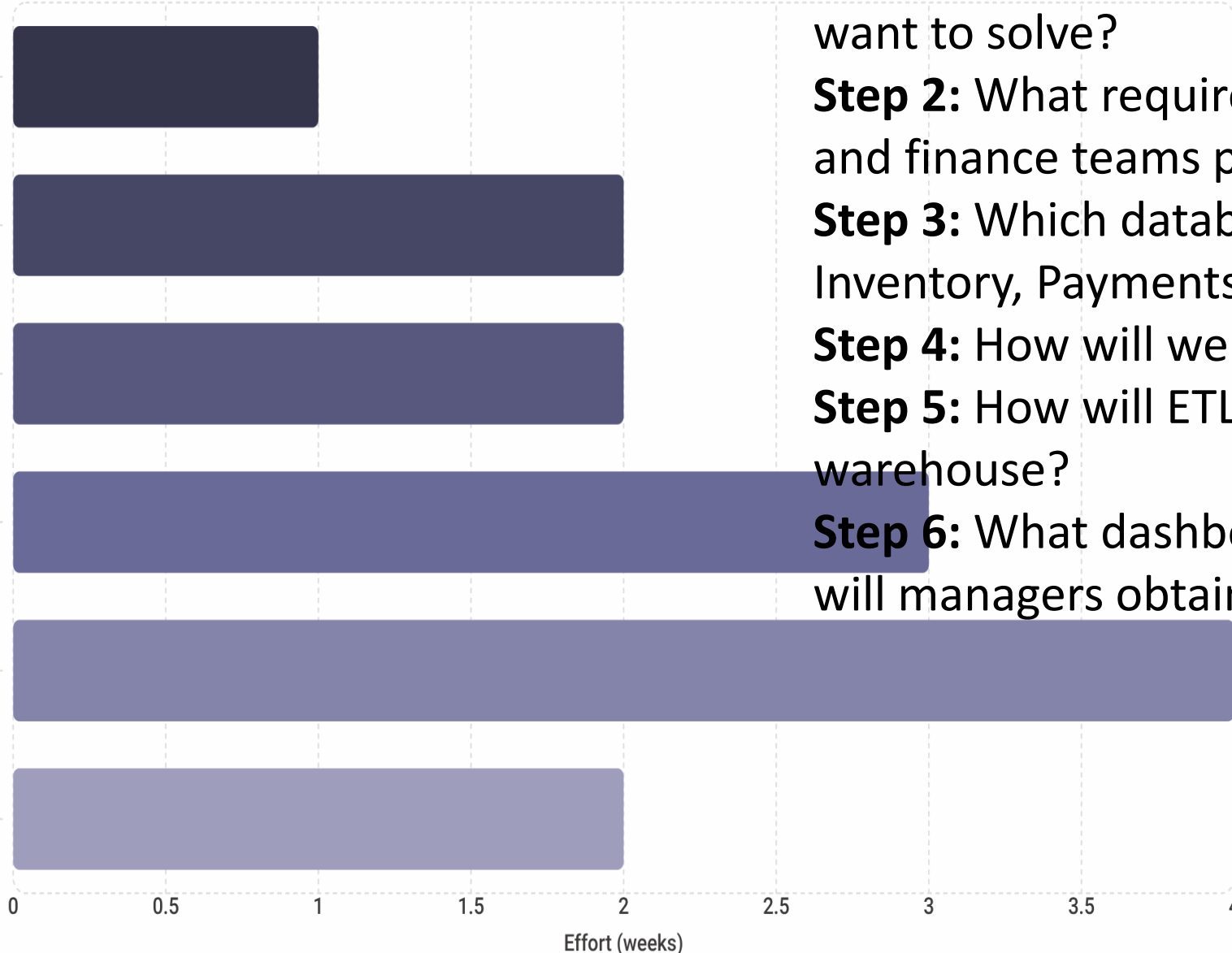
Developing a Data Warehouse — The Roadmap



Developing a Data Warehouse — The Roadmap: **Flipkart Sales Analytics**

Step

1. Business Objectives



Step 1: What business problem does Flipkart want to solve?

Step 2: What requirements will the marketing and finance teams provide?

Step 3: Which databases (Orders, Customers, Inventory, Payments) will provide the data?

Step 4: How will we design the Star Schema?

Step 5: How will ETL load the data into the warehouse?

Step 6: What dashboards and business insights will managers obtain?

Hospital DW · Requirement Gathering

The Warehouse Answers Questions — Every Question is a Requirement

Requirements are discovered by listening to what each stakeholder needs to decide. Every question maps directly to a fact table, a dimension, a measure, or a KPI.



Doctor

"Which patients with diabetes were readmitted within 30 days?"

→ **Fact:** Readmission | **Dim:** Diagnosis, Time | **KPI:** 30-day readmission rate



Finance Dept

"What is the monthly revenue trend per department?"

→ **Fact:** Billing | **Dim:** Department, Time | **KPI:** Monthly revenue



Administration

"Which wards have the highest bed occupancy rate?"

→ **Fact:** Occupancy | **Dim:** Ward, Time | **KPI:** Bed utilisation %



CEO

"How does our patient satisfaction score compare to last year?"

→ **Fact:** Survey | **Dim:** Time, Department | **KPI:** Satisfaction levels

Dimensional Modeling: Fact tables and Dimension Tables

Fact

A **Fact** is a **measurable, numerical value** that represents a business event or transaction and can be analyzed using mathematical operations such as SUM, COUNT, or AVERAGE.

Examples: Sales Amount, Quantity Sold, Profit, Discount

Dimension

A **Dimension** is a **descriptive attribute** that provides context to the facts by answering questions like **Who, What, When, Where, and How**.

Examples: Customer, Product, Time, City, Store

Flipkart Sales Analysis: Business Question: "**How much was the total sales of Laptops in Mumbai during Diwali?**"

Fact

Sales Amount (₹50,000)

Quantity Sold (2)

Discount (₹2,000)

Dimensions

Product = Laptop

Customer = Amit

City = Mumbai

Time = Diwali 2025

Dimensional Modeling: Fact tables and Dimension Tables

Imagine you visit **D-Mart** and buy groceries.

Your bill contains:

Rice – ₹500

Sugar – ₹200

Oil – ₹300

☞ **Which information can we calculate?**

Total Amount = ₹1000 ✓

Quantity = 5 kg ✓

These are **Facts** because they are measurable.

☞ **What information describes the purchase?**

Customer = Vedika

Product = Rice

Date = 28 July 2026

Store = Mumbai

Payment Mode = UPI

These are **Dimensions** because they describe the transaction.

Flipkart Sales Analytics · Dimensional Modeling

Dimensional Modeling — Turning Business Questions into Structure

Fact Table — Fact_Sales

Stores measurable events: **Revenue (₹), Units Sold, Discount Amount, Profit Margin.**
Every row = one sales transaction.

Dimensions — The 4 Ws

Customer (who) · **Product** (what) · **Location** (where) · **Time** (when). These are the lenses through which measures are analysed.

Measures vs KPIs

Measures: Raw numbers stored in the fact table (₹ Revenue, Qty). **KPIs:** Derived metrics — Conversion Rate, Average Order Value, Return Rate.

Business question → **Model:** "Which product generated the highest sales in Mumbai during Diwali?" → Fact: Sales | Dimensions: Product, Location (Mumbai), Time (Diwali period)

Multidimensional Analysis

Data Cube — Think in Three Dimensions

The Rubik's Cube Mental Model

Think of a data cube like a Rubik's Cube: each **face** = a **dimension**, each **cell** = a **stored measure**. Managers rotate the cube to view data from different perspectives — no new queries needed.

Product Axis (X)

Laptops · Phones · TVs · Appliances

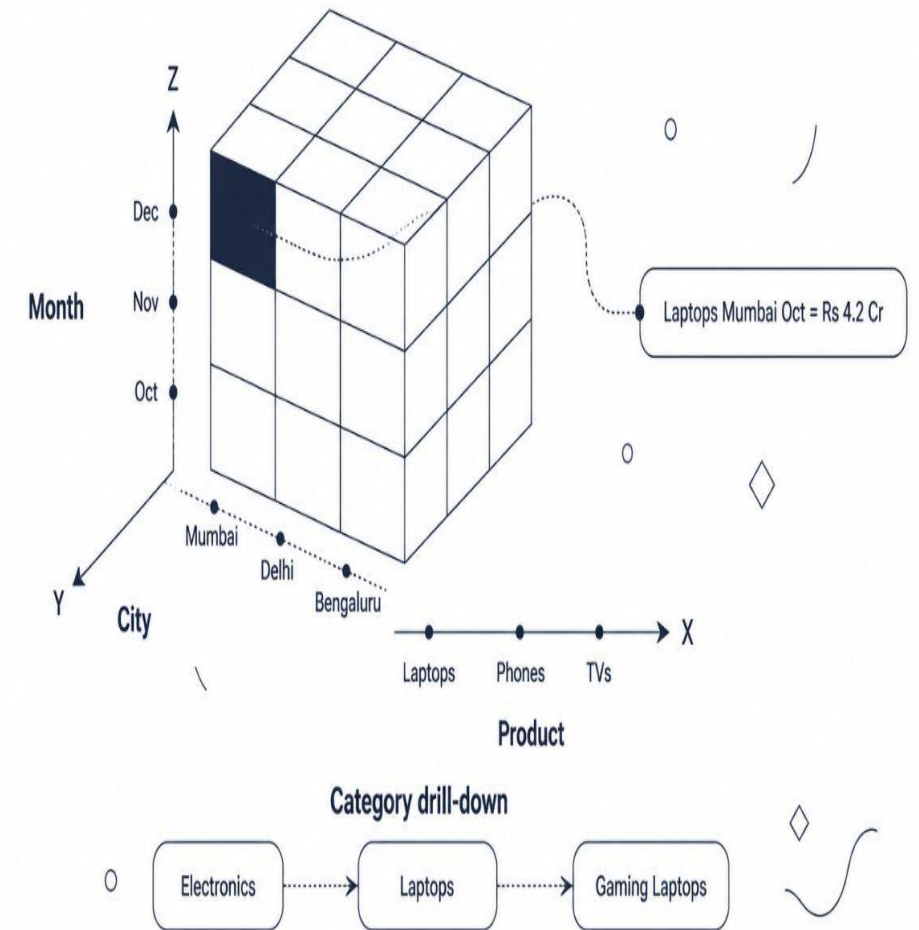
City Axis (Y)

Mumbai · Delhi · Bengaluru · Chennai

Month Axis (Z)

Oct (Big Billion Days) · Nov · Dec

📌 **Highlighted cell:** Laptops | Mumbai | October = ₹4.2 Cr
— Flipkart Big Billion Days peak sales



The **category dimension** supports drill-down: **Electronics** → **Laptops** → **Gaming Laptops**. Concept hierarchies enable managers to zoom in or out without writing new SQL.

Schema Design · Part 1

Star Schema — Simple, Fast, Powerful

The **Star Schema** is the most widely deployed dimensional model in industry. A single central fact table is joined directly to flat, denormalised dimension tables — the resulting diagram looks like a star.

Simple Joins

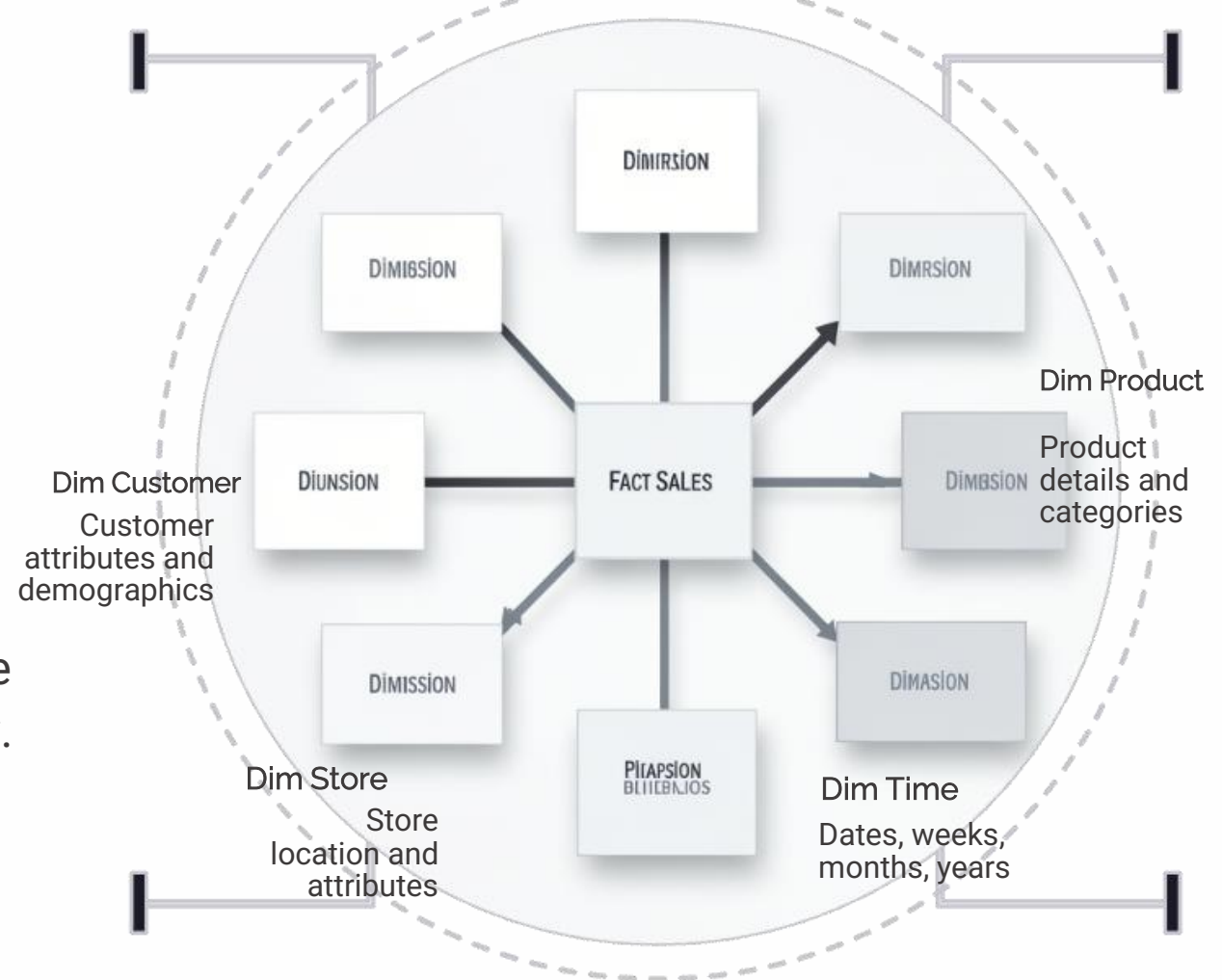
Single join from fact to each dimension. BI tools like Power BI and Tableau generate these automatically.

Fast Query Performance

Denormalised dimensions minimise join depth. Aggregation queries run significantly faster than normalised schemas.

Industry Adoption

Used by Amazon, Walmart, and most retail analytics platforms for sales and inventory reporting.

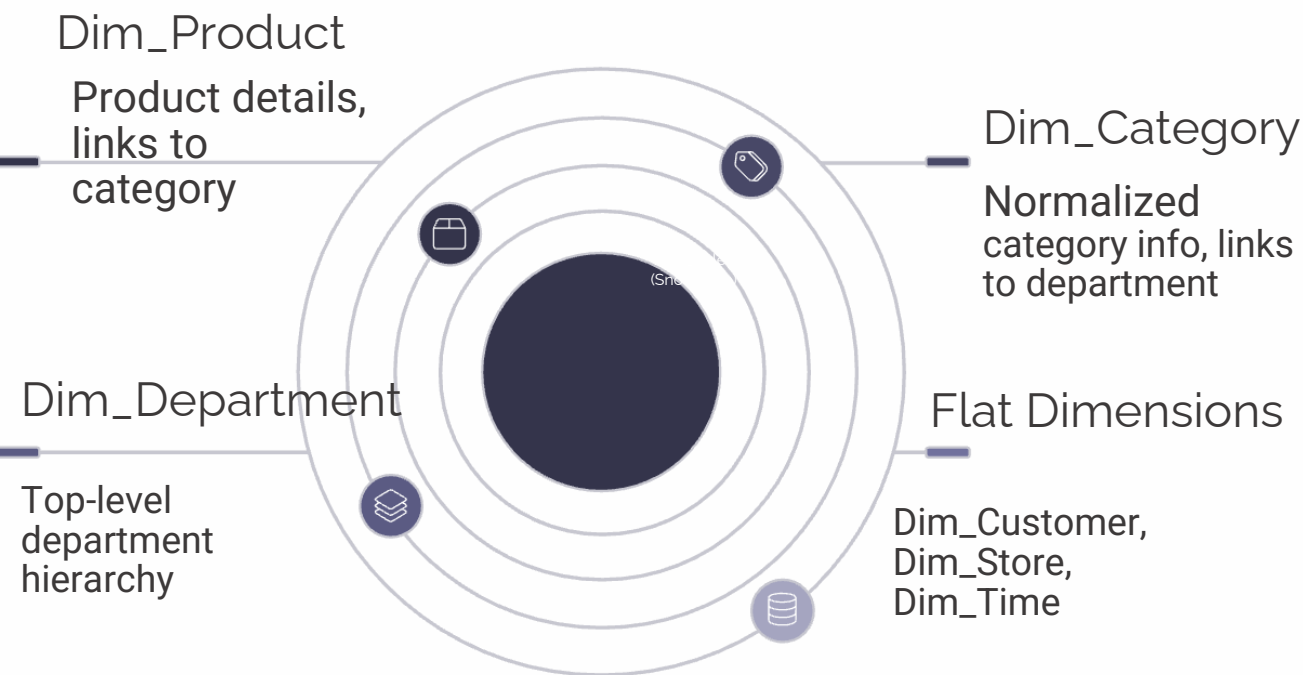


Fact_Sales (centre) → Dim_Customer · Dim_Product · Dim_Store · Dim_Time

Schema Design · Part 2

Snowflake Schema — Normalised for Accuracy

The Snowflake Schema extends the Star Schema by **normalising dimension tables into sub-dimensions**, eliminating redundancy at the cost of query complexity. Used by large retail chains like Tesco and Big Bazaar managing thousands of product categories.



Product hierarchy: Dim_Product → Dim_Category → Dim_Department

Aspect	Star Schema	Snowflake Schema
Normalisation	Denormalised	Fully normalised
Query Speed	Faster	Slower (more joins)
Storage	More space	Less space
Complexity	Simple	Complex
Best for	BI dashboards	Large, complex DWs

📱 **Disadvantage:** More complex joins slow query performance. Choose Snowflake when storage savings and data integrity outweigh query speed requirements.

Schema Design · Part 3 · Enterprise Scale

Fact Constellation Schema — Galaxy-Scale Design

Also Called: Galaxy Schema

When a single enterprise runs **multiple business processes** — orders, logistics, returns, inventory — each process gets its own fact table. Shared dimension tables connect them all.

Fact Tables

Fact_Sales · Fact_Inventory · Fact_Shipping · Fact>Returns

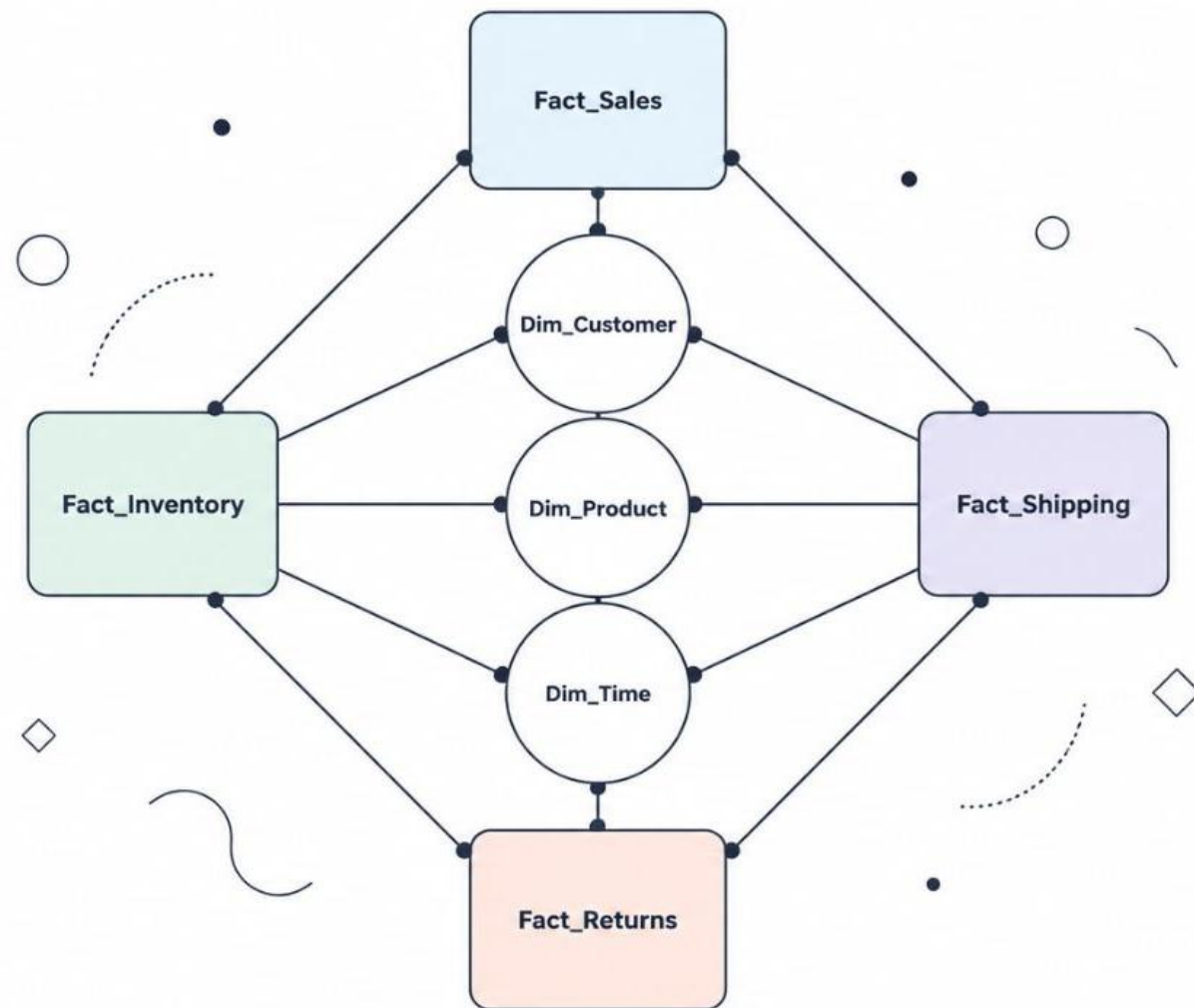
Shared Dimensions

Dim_Customer · Dim_Product · Dim_Time — used across all four fact tables

Real-World Use

Amazon's enterprise DW unifies orders, logistics, returns, and inventory in a single constellation

❏ **Trade-off:** Most flexible schema but most complex to design and maintain. Reserve for true enterprise multi-process environments.



Knowledge Discovery · DMQL_Data Mining Query Language

DMQL — Beyond Retrieval, Towards Discovery

SQL retrieves data. DMQL *discovers knowledge* — patterns, rules, and predictions hidden within the warehouse. It operates on data cubes and fact tables to surface actionable intelligence.

- Classification
"Which customers are likely to churn?" → Label each customer **High / Medium / Low risk** for targeted retention campaigns.
- Association (Market Basket)
"Customers who buy iPhones also buy AirPods **78% of the time**" → Bundle recommendations and placement decisions.
- Clustering
Group customers by purchasing behaviour **without predefined labels** — discover natural segments such as value shoppers vs impulse buyers.
- Prediction
"Forecast next quarter's sales for premium smartphones in Tier-1 cities" → Drive inventory and marketing budget allocation.

Feature	SQL	DSQ
Purpose	Retrieve stored data	Discover hidden patterns
Output	Rows & columns	Rules, clusters, predictions
Example query	<pre>SELECT * FROM Sales WHERE city='Mumbai'</pre>	<pre>MINE patterns FROM Sales IN CUBE WHERE category='Electronics'</pre>
Used for	Reporting & dashboards	Machine learning / Data mining

DMQL operates on **data cubes**, not flat tables — enabling pattern discovery across multiple dimensions simultaneously. This is what separates a data warehouse from a simple database.